Testarea Sistemelor Software
-Etapele 1 și 2-

REZOLVAREA ECUAȚIEI DE GRADUL II

Universitatea din București
Facultatea de Matematică și Informatică

BILIUȚĂ Elisa-Adelina(311)
MARCOVICI Anamaria(311)
NEDELESCU George-Eduard(311)

Scopul proiectului

- Implementarea unui solver pentru ecuații de gradul II
- Testarea corectitudinii programului
- Aplicarea metodelor de testare
- Crearea mutanților(TBC)
- Testarea pe mutanți (TBC)
- Concluzii

Problema noastră

- Rezolvare ecuației $ax^2 + bx + c = 0, a, b, c \in \mathbb{R}, a \neq 0$
- Calcularea discriminantul $\Delta = b^2 - 4ac$
- În funcție de discriminant vom avea cazurile:

$$\begin{cases} \Delta > 0 \text{ , două soluții reale distincte} : x_{1,2} = \dfrac{-b \pm \sqrt{\Delta}}{2a} \\ \Delta = 0 \text{ , două soluții reale egale} : x_1 = x_2 = -\dfrac{b}{2a} \\ \Delta < 0 \text{ , două soluții complexe} : x_{1,2} = \dfrac{-b \pm i\sqrt{-\Delta}}{2a} \end{cases}$$

Implementare

- Limbajul utilizat: Python
- Clasa: EquationSolver
- Metoda solve()

```python
class EquationSolver:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def solve(self):
        if not all(isinstance(x, (int, float)) and not isinstance(x, bool) for x in [self.a, self.b, self.c]):
            raise ValueError("Coeficientii ecuatiei trebuie sa fie numere reale!")

        if self.a == 0:
            raise ValueError("Coeficientul a nu poate fi 0!")

        delta = self.b ** 2 - 4 * self.a * self.c

        if delta < 0:
            real_part = -self.b / (2 * self.a)
            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
            x1 = complex(real_part, imag_part)
            x2 = complex(real_part, -imag_part)
            return x1, x2

        elif math.isclose(delta, 0.0, abs_tol=1e-9):
            return -self.b / (2 * self.a)

        else:
            x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
            x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
            return x1, x2
```

Testarea funcțională

- Black Box Testing
- Testare bazată pe input/output
- Fără analiză de cod
- Scop: testăm comportamentul programului fără a ne uita deloc la implementare

Clase de echivalență(CE)

- Datele invalide vor genera erori
- Dacă a este nul, vom avea eroare
- $\Delta > 0 \rightarrow$ 2 soluții reale distincte
- $\Delta = 0 \rightarrow$ 2 soluții reale egale
- $\Delta < 0 \rightarrow$ 2 soluții complexe

Clase de echivalență -rezultate așteptate-

ID Test	Clasa	a	b	c	Δ	Rezultat așteptat
TC_CE_01	C_{E_1}	"text"	2	1	N/A	$\boldsymbol{E_1}$ (Excepție: tipuri invalide)
TC_CE_02	C_{E_2}	0	2	1	N/A	$\boldsymbol{E_2}$ (Excepție: a nu poate fi 0)
TC_CE_03	C_{S_1}	1	1	1	-3	$\boldsymbol{S_1}$ (Două rădăcini complexe)
TC_CE_04	C_{S_2}	1	2	1	0	$\boldsymbol{S_2}$ (Rădăcină reală dublă)
TC_CE_05	C_{S_3}	1	5	6	1	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)

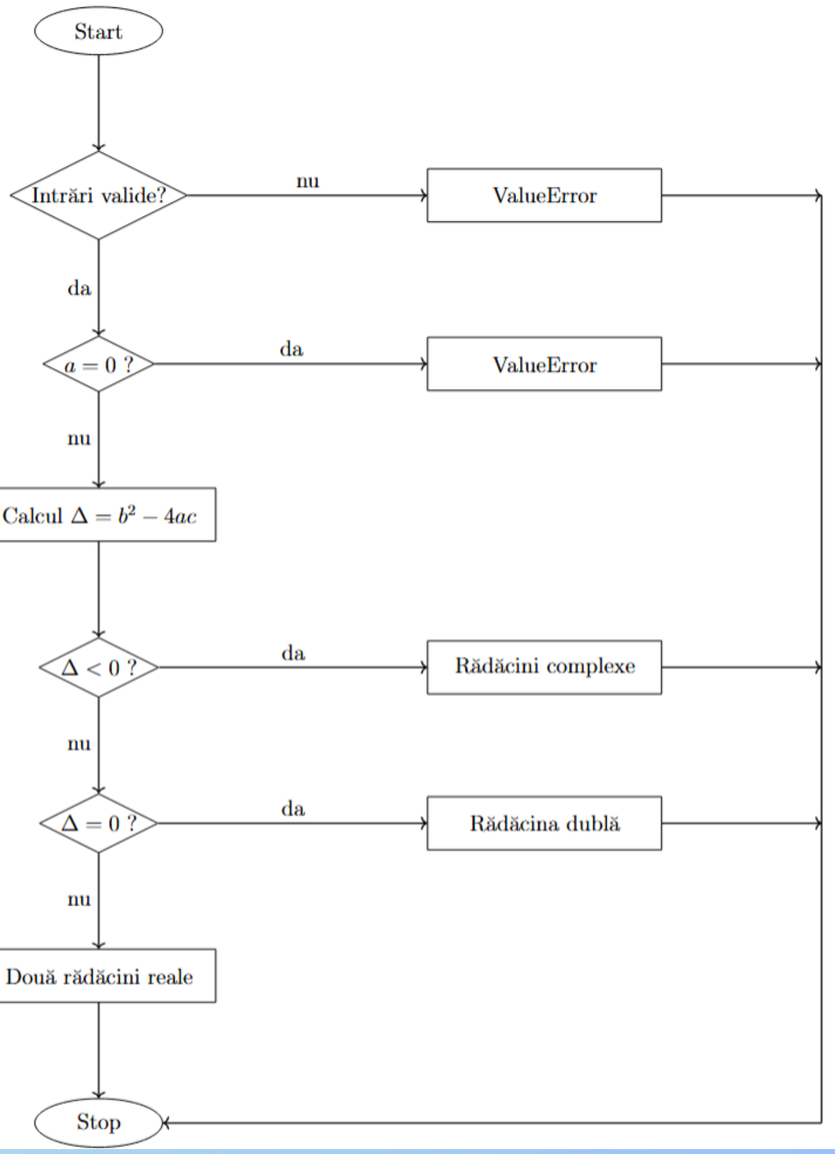
Boundary testing (AVF)

- Testăm valorile critice
- $a = 0$
- $\Delta = 0$

ID Test	Frontieră	a	b	c	Δ	Rezultat așteptat
TC_CE_02	$a = 0$	0	2	1	N/A	$\boldsymbol{E_2}$ (Excepție: a nu poate fi 0)
TC_AVF_01	$a \to 0^-$	-0.001	2	1	4.004	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)
TC_AVF_02	$a \to 0^+$	0.001	2	1	3.996	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)
TC_CE_04	$\Delta = 0$	1	2	1	0	$\boldsymbol{S_2}$ (Rădăcină reală dublă)
TC_AVF_03	$\Delta \to 0^-$	1	1.999	1	≈ -0.004	$\boldsymbol{S_1}$ (Două rădăcini complexe)
TC_AVF_04	$\Delta \to 0^+$	1	2.001	1	≈ 0.004	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)

Testare structurală

- White Box Testing
- Analiză de cod
- Control Flow Graph



Coverage

- Statement coverage
- Branch coverage
- Toate cazurile sunt acoperite și testate

Condiție	Evaluare True	Evaluare False
`not all(...)`	TC_CE_01, TC_PC_01–03	TC_CE_02–05, TC_PC_05–11
`a == 0`	TC_CE_02, TC_PC_04	toate testele valide
`delta < 0`	TC_CE_03, TC_PC_10, TC_AVF_03	TC_CE_04–05, TC_PC_08–09, TC_AVF_01–02–04
`isclose(delta, 0)`	TC_CE_04, TC_PC_08	cazurile cu $\Delta < 0$ și $\Delta > 0$

Tabela 3.1: Acoperirea condițiilor pentru metoda `solve()`

Complexitate

- Complexitatea ciclomatică McCabe=4 (calculată folosind ecuația V(G) = E – N + 2P = D + 1 = 3+1 , unde

E = numărul de muchii ale grafului de flux de control,

N = numărul de noduri ale grafului,

P = număul de componente conexe (P=1),

D = numărul de decizii(3 deoarece am verificat coeficienții să fie de tip float/int, am verificat condiția a=0 și am evaluat discriminantul)

Concluzii

- Program corect testat
- Acoperire completă
- Metode eficiente
- Rezultatele confirmă că aplicația funcționează corect

```
"D:\FMI\ANUL III\TESTAREA SISTEMELOR SOFTWARE\Proiect TSS\.venv\Scripts\
    python.exe" "D:/SOFTURI/PyCharm 2025.3.3/plugins/python-ce/helpers/
    pycharm/_jb_unittest_runner.py" --path "D:\FMI\ANUL III\TESTAREA
    SISTEMELOR SOFTWARE\Proiect TSS\test_equation_solver.py"
Testing started at 17:02 ...
Launching unittests with arguments python -m unittest D:\FMI\ANUL III\
    TESTAREA SISTEMELOR SOFTWARE\Proiect TSS\test_equation_solver.py --quiet
     in D:\FMI\ANUL III\TESTAREA SISTEMELOR SOFTWARE\Proiect TSS

Ran 22 tests in 0.024s

OK

Process finished with exit code 0
```

Vă mulțumim pentru atenție!

BILIUȚĂ Elisa-Adelina(311)
MARCOVICI Anamaria(311)
NEDELESCU George-Eduard(311)